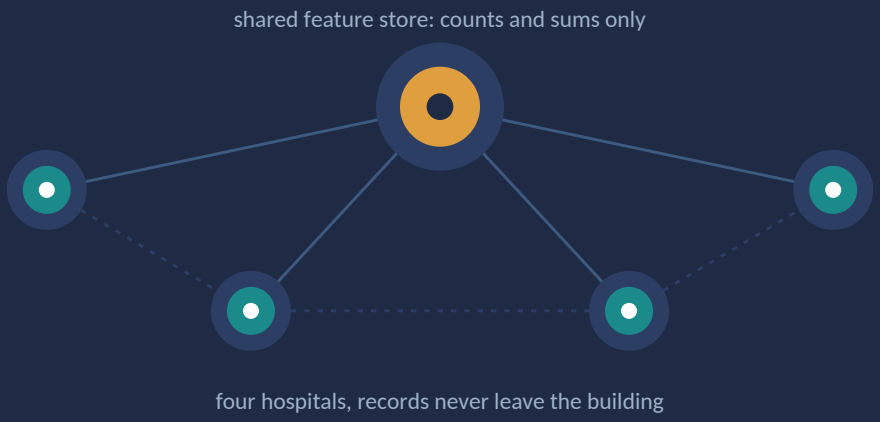
STUDY GUIDE | NOVEMBER 2026

Understanding the project from scratch

Plain words first, details second. Read it in order, then say each chapter out loud without looking.



12
short chapters, one idea each

0
prior knowledge assumed

Say it
every chapter ends with a test you can run on yourself

For Harshita Pandita and Pranjal Tomar

Companion to the repository github.com/hersheyharshi/federated-feature-store
Every number here comes from results/summary.md in that repository.

BEFORE YOU START

How to use this guide

This guide explains the whole project in the order it makes sense, not the order the code was written. Each chapter has the same shape: the idea in plain words, how our project uses it, the numbers we found, and one thing to say out loud. If you cannot say it, read the chapter again before moving on.

Chapters	What you will be able to do afterwards
1 to 3	Tell the story, read a results table without being fooled, and explain federated averaging with three numbers.
4 to 7	Explain the feature store, differential privacy, drift and forgetting, and the Switzerland fix.
8 to 10	Say which claims are proven and which are only measured, find your way around the code, and rerun everything.
11 and 12	Answer twelve hard questions, and use the glossary.

> **A rule worth keeping**
> If one of you cannot explain a part, you are not done with it. Take turns explaining chapters to each other, and let the listener interrupt with "why?" at least three times.

Suggested pace: one chapter per sitting, about twenty minutes each. The whole guide is an afternoon, spread over two days if you want it to stick.

CHAPTER 1

The story in one page

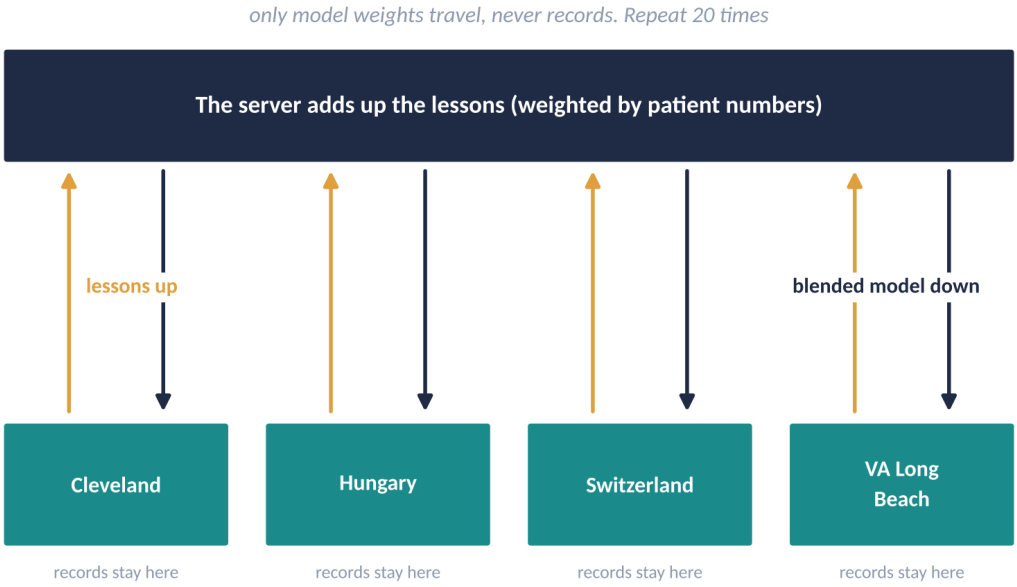
Picture four schools that each want a better exam-prep booklet. They are not allowed to show each other their students' marks. So each school tries the booklet on its own students, notes what should change, and sends *only the changes* to a coordinator. The coordinator combines the four sets of changes into one improved booklet and sends it back. They repeat that a few times, and every school ends up with a booklet shaped by all four, without anyone seeing another school's marks.

In the story	In our project
A school	A hospital (Cleveland, Hungary, Switzerland, VA Long Beach)
Students' marks	Patient records. They never leave the hospital.
The booklet	The model that predicts heart disease
"What should change"	The model's weights, a list of numbers
The coordinator	A server that averages the weights

That is federated learning. On its own it leaves three problems, and each one is a part of the project:

- **The changes themselves can leak information** about individual students. Fix: differential privacy, measured noise (chapter 5).
- **The schools might measure things differently.** One uses centimetres, another inches. Fix: a shared feature store, one agreed set of definitions (chapter 4).

- **The world changes.** A new teaching method changes what works, and learning the new can wipe out the old. Fix: drift detection and memory protection (chapter 6).



The loop: lessons go up, a blended model comes back, records never move.

> **Say it out loud**
> Four hospitals cannot share records, so each trains at home and sends only lessons. A server blends them. We then added privacy, shared definitions, and a way to cope with change.

CHAPTER 2

The data, and the accuracy trap

We use the public UCI Heart Disease data, which came from four real hospitals. We treat each one as a separate hospital in our simulation. Each patient has 13 facts (age, blood pressure, cholesterol, chest pain type and so on) and a yes or no label: does this person have heart disease?

Hospital	Patients	Share with heart disease
Cleveland	303	46%
Hungary	294	36%
Switzerland	123	93%
VA Long Beach	200	74%

Practice patients and test patients

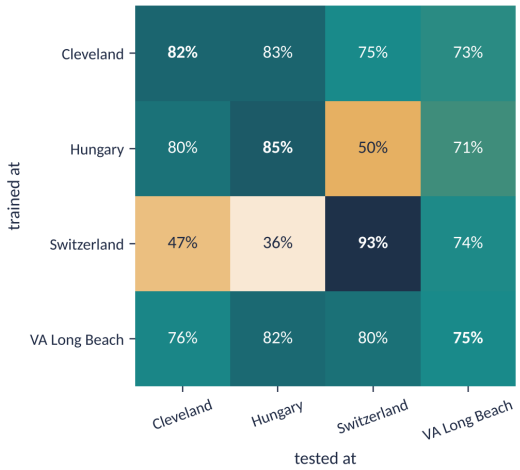
At every hospital we keep 70% of patients for practice and hide 30% as a test pile that the model never sees. Grading a model on the patients it practised on would only measure memory, like marking a student on the exact questions they rehearsed.

Accuracy can lie

At Switzerland 93% of patients are sick. A lazy "model" that says **sick** for everybody would be right about 94% of the time without learning anything. So we always print the **lazy guess** next to our results. A score only means something if it beats it. Across the four hospitals the lazy guess averages 71.9%.

AUC: a fairer score

AUC asks a different question: if you pick one sick and one healthy patient at random, how often does the model give the sick one the higher risk? 0.5 is a coin flip, 1.0 is perfect. It cannot be fooled by the base rate, but with very few healthy patients it gets noisy, which is why Switzerland's AUC jumps around.



Each model tested at every hospital. Look at the Switzerland row.

> **Say it out loud**
> We hide a test pile because practice scores prove nothing. We print the lazy guess because accuracy is meaningless without it.

CHAPTER 3

Federated averaging with three numbers

Strip a model down to one weight and the whole federated idea fits on a napkin. Suppose three hospitals start from the same weight, train for a bit on their own patients, and come back with different answers:

Hospital	Patients	Weight after local training
A	100	2.0
B	50	4.0
C	50	6.0

A plain average of the three would be 4.0. But hospital A has twice as many patients, so we weight each answer by its patient count:

> (100 x 2.0 + 50 x 4.0 + 50 x 6.0) / 200 = (200 + 200 + 300) / 200 = **3.5**

That 3.5 goes back to all three hospitals as the new starting point, and they go round again. Our real model has 14 weights for the logistic regression (13 inputs and a bias) and a few hundred for the small neural network, but the arithmetic is the same.

Our settings

20 rounds, 2 local passes over the data per round, mini-batches of 32 patients. We also trained a **pooled** model on all 920 patients together. That is illegal in real life, but it shows the best we could hope for: our **ceiling**. The **baseline** is what you get if you do nothing clever, each hospital on its own.

Why uneven hospitals make it harder

Switzerland is small and mostly sick; Hungary is mostly healthy. Their lessons pull in different directions, and averaging them is a compromise. That tension runs through every result in this project.

> **Say it out loud**
> Hospitals train locally, the server takes a weighted average of their weights, sends it back, and they repeat. Draw it with the numbers 2, 4, 6 and the answer 3.5.

CHAPTER 4

The shared feature store

A **feature** is one input to the model, such as cholesterol. A **feature store** is one agreed, versioned description of every input: its name, meaning, unit, valid range and version number. Without it, one hospital could record cholesterol as 233 (mg/dl) and another as 6.0 (mmol/L) for the same kind of patient, and the shared recipe turns into nonsense.

What ours contains

- A schema of the 13 inputs, version 1.0, in **feature_store.py**. Impossible values become "missing": a cholesterol of 0, for instance, means "not recorded", so it is removed.
- Shared summaries. Each hospital computes, from its own records, a count, a sum and a sum of squares for every input. Only those numbers travel. The server turns them into one shared average and spread per input, and every hospital uses them to scale its data.
- Missing values are filled with the shared average, the same way everywhere.

> **What is shared and what is not**
> Shared: the schema, and three summary numbers per input. Never shared: any patient record. Note that these summaries are **not** protected by differential privacy by default.

What difference it makes

If every hospital scales its inputs using only its own average and spread, then "a value of 1.0" means something different at each one, and the averaged model is confused. In our results this single change moves federated accuracy from 72.6% to 81.0%, a gain of 8.4 points, and it was better in all ten runs. It was the biggest effect we found.

The messy reality of the data

	What is missing (after cleaning)
Switzerland	Cholesterol entirely; fasting sugar 61%; vessel count 96%
Hungary	Vessel count 99%; thalassemia 90%; ST slope 65%
VA Long Beach	Vessel count 99%; thalassemia 83%; about 28% of vital signs

So two inputs carry real information almost only at Cleveland. We keep that in mind whenever Switzerland looks odd.

> **Say it out loud**
> The feature store is one shared list of what every input means, with a version number. Only definitions and three summary numbers per input travel, never records.

CHAPTER 5

Differential privacy and epsilon

Even the lessons a hospital sends can leak a hint. If one patient is unusual, the lesson might point at them. Differential privacy fixes this with two moves, applied at each training step inside each hospital:

- **Clip.** Limit how far any single patient can move the model. Nobody gets an outsized say.
- **Blur.** Add random noise of a carefully chosen size to what remains.

The promise

With a privacy budget of **epsilon**, adding or removing any one patient from a hospital changes the chance of any outcome by at most a factor of e to the power epsilon, plus a tiny slack called delta (we use one in 100,000). For epsilon 1 that factor is about 2.7: the released models would look almost the same with or without you. **Smaller epsilon means more noise, more privacy, less accuracy.**

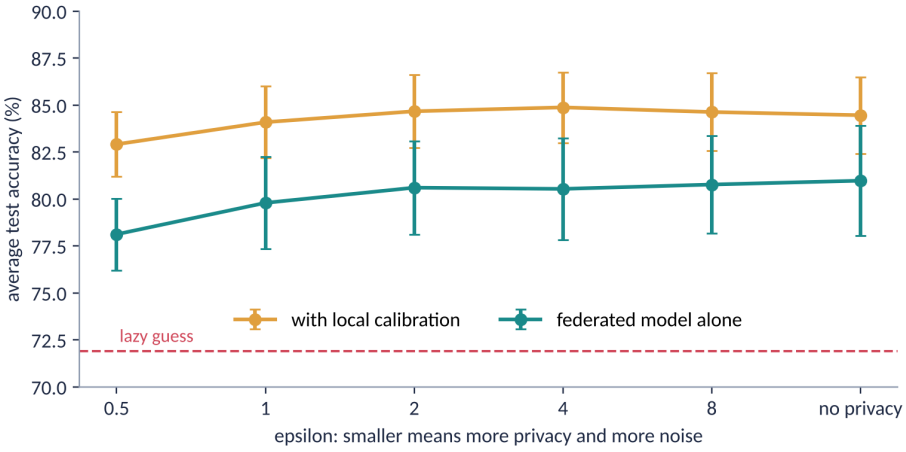
The accountant

Every training step spends a little privacy. An **accountant** adds up the total over all steps and rounds. We choose the noise so the total comes out at the epsilon we want. We wrote our own accountant, and checked it reproduces a published reference value (3.01) in a unit test.

What it cost us

Epsilon	0.5	1	2	4	8	none
Logistic regression	78.1%	79.8%	80.6%	80.5%	80.8%	81.0%
Change vs none (paired)	-2.9	-1.2	-0.4	-0.4	-0.2	

From epsilon 2 up we cannot tell it from no privacy. The small neural network pays more (3.0 points at epsilon 1, 5.5 at 0.5), probably because it has far more weights to protect.



What our guarantee does not cover

Only training is protected. The shared feature-store summaries are not. We tried adding noise to them too, and accuracy collapsed to 58-62%, below the lazy guess, unless the budget was 100, which protects almost nothing. That is a negative result and we report it. Replay and EWC were also run without privacy.

The membership test

As a sanity check we ran a simple attacker that guesses whether a patient was in the training set by looking at how confident the model is about them. For logistic regression it scored 0.51, a coin flip, with or without privacy: the model barely memorises. For the neural network, which memorises more, the attacker got 0.57 without privacy and 0.50 with it. This measures one simple attacker; the guarantee comes from the accounting, not from this test.

Say it out loud

Epsilon is the privacy dial. Smaller means more noise, more privacy, less accuracy. Epsilon 1 means any one patient changes any outcome's probability by at most a factor of about 2.7.

CHAPTER 6

Drift, forgetting and protection

Concept drift

Concept drift is when the rule linking inputs to outcomes changes. A new treatment arrives and patients who used to be at high risk now mostly recover. The patients look the same; what their symptoms mean has changed. (When the patients themselves change, that is *data drift*, a different thing.)

How we simulated it

We split every hospital's patients at random into an early stage A and a later stage B. In stage B we flip the label for sick patients with a high value of the strongest risk marker (usually ST depression) so that most of them become "recovered". We tried three strengths: 30%, 60% and 90% of those patients recover. Only the rule changes; the inputs do not.

The alarm

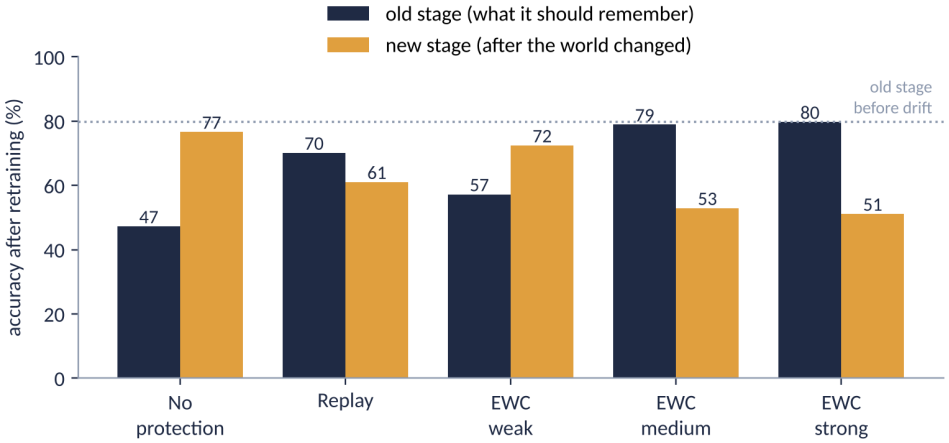
We watch the mistakes of the frozen stage-A model on a stream of patients. The detector (DDM) raises an alarm when the running error rate climbs well above its best level so far. At the strongest drift it fired in 8 of 10 runs, a median 27 patients after the change; at the gentlest, in only 3 of 10. It also raised a false alarm before the drift in 2 of 10 runs. So detection works on strong change and struggles on gentle change.

Catastrophic forgetting

When a model keeps training on new data it can lose what it knew. Like cramming for a new exam and forgetting the last one, but far more severe. Unprotected, our model fell from about 80% to 47% on the old patients, a loss of 33 points.

Two protections

- **Replay.** While learning stage B, each hospital also rehearses a sample of its own stage-A patients. They never leave the hospital.
- **EWC** (elastic weight consolidation). Work out which weights mattered most for the old task and make them expensive to change. A strength setting, lambda, controls how expensive.



Navy: still remembers the old stage. Amber: learned the new stage.

The trade-off

Protection costs new learning. Strong EWC almost never forgets (0.1 to 0.8 points) but its new-stage accuracy stays near 51-53%, because it barely moved. Replay forgets 10 points and learns the new rule to 61%. On the average of both stages every protection beats doing nothing, but their differences are within the noise, so we do not rank them. For the neural network, replay is clearly the best option.

> **Say it out loud**
> Drift means the rule changed; forgetting means learning the new erased the old. Replay and EWC protect the old, but at a cost to the new, and a low forgetting number alone is misleading.

CHAPTER 7

The Switzerland problem and the local offset

After the feature store, one problem remained. A single shared model scored 77.3% at Switzerland while a model trained only there scored 93.2%. The reason is simple: Switzerland is 93% sick, a local model learns that, and a global model trained mostly on healthier hospitals cannot.

The fix is tiny. After training, each hospital looks at the shared model's output on **its own practice patients** and fits **one number**, an offset that shifts the model's decision so its predicted sick rate matches that hospital's real one. The offset never leaves the hospital, and test patients are never used.

At Switzerland	Accuracy
Federated model alone	77.3%
Federated + local offset	93.0%
Local model at home	93.2%
Lazy guess (always "sick")	94.6%

The offset added 15.7 points at Switzerland (95% interval 10.9 to 20.5) and made federated learning match local models on average (+0.5 points, interval 0.0 to 1.0).

The honest caveat
The calibrated 93.0% is still below the lazy guess of 94.6%. The offset fixes the base-rate gap; it does not make the model rank Switzerland's patients better (AUC about 0.6, and with only about 37 test patients, about 2 of them healthy, that number is very noisy). Do not oversell it.

Say it out loud
A global model cannot know a hospital's base rate, so each hospital fits one local number to match its own. It closes the accuracy gap, but it is not better medicine.

CHAPTER 8

Reading the results honestly

Ten seeds, and what that means

A **seed** fixes the random choices: which patients land in the practice pile, and the randomness inside training. We ran everything with ten different seeds, then compared methods **seed by seed** (a paired comparison), which is much sharper than comparing two averages. We report the average difference and a 95% interval around it. If that interval excludes zero, the difference is probably real; if it straddles zero, we say we cannot tell.

One limit: the ten seeds re-split the *same 920 patients*. The intervals capture split-and-training randomness, not what would happen with new patients.

Proven or measured?

Claim	Status
Each hospital's released models are (epsilon, delta)-private for adding or removing one patient, under stated assumptions	**Proven** (standard accounting)
Error after drift is at most the old error plus the fraction of labels changed, when inputs do not shift	**Proven** for our simulation; close to an identity there, so only a sanity check
Accuracy at each epsilon; federated versus pooled; the feature-store gain	Measured
Forgetting, recovery and alarm delay under simulated drift	Measured
Anything about real hospitals or clinical use	Not claimed

Things we do not claim

- That nobody could ever be identified under any attack. The guarantee is mathematical and conditional.
- That the drift bound proves safety in real hospitals. It does not.
- That the model is ready for clinical use. It is a simulation on public data.

> **Say it out loud**
> A guarantee is proven under stated assumptions; a measurement is what we saw in these experiments. We mark every claim as one or the other.

CHAPTER 9

A map of the code

Everything lives in **src/fedfs/**. Read the files in this order and you will meet the ideas in the order of this guide.

File	What it does	Names to look for
data.py	Loads the four hospitals and splits each 70/30	load_uci, split_hospitals
feature_store.py	The schema, cleaning, shared summaries, scaling	SCHEMA, local_stats, FeatureStore
models.py	Logistic regression and the small network; local offset	Logistic, MLP, fit_offset
federated.py	Local training, federated averaging, replay and EWC	local_train, fedavg, EWC
privacy.py	The privacy accountant and noise calibration	epsilon, noise_for_epsilon
drift.py	Simulated drift and the DDM alarm	simulate_drift, DDM, detect
experiments.py	Every experiment, plus the paired-comparison helper	run_baselines, run_drift, paired
run_all.py	Runs everything and writes results/ and summary.md	main
plots.py	The figures for the report	plot_privacy_curve

Other places

- **tests/test_fedfs.py**: 14 tests, including a check of the accountant against the published value and a numerical check of the network's gradients.

- **docs/theory.md**: the proven-versus-measured statements and their assumptions.
- **docs/report.md** and **docs/report.pdf**: the full write-up.
- **results/**: every table and figure, one folder per model.

> **Say it out loud**
> Pick any one file and explain what goes in, what comes out, and which chapter of this guide it belongs to.

CHAPTER 10

Run it yourself

Doing this once, yourselves, is what makes the work yours. It also confirms the numbers in this guide.

Step	What to do
1	Open a new Colab notebook. Upload the repository zip with the folder icon, then run: **!unzip -q federated-feature-store.zip** **%cd federated-feature-store**
2	A quick check first (a few minutes): **!python -m fedfs.run_all --download --seeds 3 --out check**
3	The full run (roughly 20 to 40 minutes): **!python -m fedfs.run_all --download --seeds 10 --out results**
4	Open **results/summary.md** and compare with this guide and the report. Tiny differences from library versions are fine. Large ones are not: tell someone.
5	Read the tables before the figures. For each table, say aloud what the best and worst rows mean.

> **If something looks off**
> Do not paper over it. Write what you see in the Results Log, with the date and the exact command. A recorded surprise is worth more than a clean story.

CHAPTER 11

Twelve hard questions

Practise these out loud. The short answers below are a floor, not a script; say them in your own words.

Question	Short answer
1. What does epsilon 1 mean?	Adding or removing any one patient changes the probability of any outcome by at most a factor of about 2.7, plus a tiny delta. Smaller epsilon is more private and less accurate.
2. Why a shared feature store?	Without agreed definitions and scaling, the same number means different things at different hospitals. It lifted accuracy by 8.4 points, in 10 of 10 runs.
3. Why does federated learning lose at Switzerland, and how did you fix it?	One global model cannot learn a 93% sick base rate. A single offset per hospital, fitted locally, matched local accuracy (93.0% versus 93.2%).
4. Does the offset leak anything?	It is fitted on the hospital's own practice patients and never leaves; nothing new is released.

Question	Short answer
5. What is proven and what is only measured?	The training privacy guarantee and the drift bound (for our simulation) are proven under assumptions. Every accuracy, forgetting and alarm number is measured.
6. Why does the drift bound need the inputs unchanged?	It bounds the change in the label rule only. If the patients also shift, an extra term for that shift is needed.
7. Why is low forgetting from EWC misleading?	Strong EWC barely moves, so it keeps the old and fails to learn the new (about 51%). Look at both stages.
8. Why did private summaries fail?	The noise scale follows the schema ranges (for example cholesterol up to 700), which dwarfs the true sums. Accuracy fell below the lazy guess.
9. What do ten seeds and the intervals tell you?	Differences are real if the paired 95% interval excludes zero. They reflect split and training randomness over the same 920 patients, not new patients.
10. How do you know the accountant is right?	We wrote it ourselves and a unit test reproduces a published reference value (3.01).
11. What would you do next?	Per-hospital calibration inside training, tighter bounds so private summaries can work, gentler drift, and real hospital data with proper access.
12. Is it clinically useful?	No. It is a simulation on public data with drift we created, and it has not been validated clinically.

CHAPTER 12

Glossary

Term	In plain words
Federated learning	Hospitals train at home and share only what the model learned.
Weights	The list of numbers that is the model.
Baseline / ceiling	The number to beat, and the best case if data could be pooled.
Lazy guess	Always answering the more common label. The accuracy to beat.
AUC	Chance of ranking a sick patient above a healthy one. 0.5 is a coin flip.
Feature store	One shared, versioned list of what each input means and how it is prepared.
Differential privacy	Measured noise so no single patient can be picked out.
Epsilon	The privacy dial. Smaller means more privacy and more noise.
Delta	A tiny slack in the privacy promise (we use 1 in 100,000).
Clipping	Limiting how much one patient can move the model.
Accountant	Adds up the privacy spent over every step.
Concept drift	The rule linking inputs to outcomes changes.
Catastrophic forgetting	Learning new things erases the old.
Replay	Rehearsing some old examples while learning new ones.
EWC	Making the weights that mattered for the old task expensive to change.
Seed	A number fixing the random choices, so a run can be repeated.
Paired comparison	Comparing two methods on the same seed, then averaging the differences.
Local offset	One number a hospital fits so the model matches its own base rate.